

Tool Routing - Take-Home Exercise

Background / Overview

Consider a general-purpose agent, similar to the Claude consumer application, in which a user may connect arbitrary MCP servers to extend the agent's capabilities. Each server contributes a set of tools. A user who connects several servers can easily accumulate several hundred tools, most of which are irrelevant to any particular request.

Passing the full tool set to the model on every turn is expensive and counterproductive. It increases prompt size, latency, and cost, and it degrades the model's ability to select the correct tool: selection accuracy falls as the number of available tools rises. Three common servers, PostHog, Linear, and GitHub, together expose more than one hundred tools.

The task is to build a tool router: a component that, given a user query and the currently connected tools, selects a small relevant subset before the agent reasons over it. You will build the router, a minimal harness to exercise it, and an evaluation that measures whether it works.

The connected servers are not known in advance. In production a user may connect any server, so the router has access only to the metadata each server advertises for its tools - name, description, and parameter schema. It must generalize across servers it has never seen. DO NOT rely on hardcoded server categories, curated routing tables, or per-tool tuning.

Scope

In scope:

- A tool router that narrows an arbitrary set of connected tools to a relevant subset for a given query.
- A minimal harness that exercises the router end to end. A loop that calls the model, executes any requested tool calls, and repeats until completion is sufficient, provided the router runs within it and narrows the tool set before each model call.
- An evaluation that measures routing quality.

Out of scope:

- Any harness capability beyond the loop above: memory, multi-turn session handling, retries, streaming, error recovery, planning, or a user interface.
- Training or fine-tuning a model. Use off-the-shelf models.
- Connecting to live MCP servers, deployment, or auth plumbing. Hardcode auth or mock tools for your submission as you see fit. Remember that the tool router needs to be agnostic of tools so any mocking must be limited to tool calls only
- Cost optimization - prefer correctness over cost.

Example Scenario

As an example, a user might connect the following three servers, which together expose more than one hundred tools:

Server	Domain
GitHub	repositories, pull requests, issues, CI, code search
PostHog	product analytics, feature flags, experiments, error tracking, SQL
Sentry	error monitoring, issues, events, releases, root-cause analysis

This is only an illustration; you are not restricted to these servers. Note that such a set reflects real conditions:

- capabilities that overlap across servers (PostHog and Sentry both surface errors; GitHub and Sentry both call grouped work "issues"),
- tools whose correct use depends on a capability one server has and another lacks (a GitHub issue can be created, whereas a Sentry issue is created automatically from errors and can only be updated),
- uneven or terse tool descriptions, and
- a large majority of tools irrelevant to any given query.

Some sample queries:

1. What's broken in the latest build on `main` in `acme/api`?
2. Signups are way down this week, can you dig into why?
3. Checkout's been throwing errors since we shipped #1242, figure out what's going on.
4. We keep seeing that same crash in prod. is it already tracked anywhere, and if not, file it.

You should decide what constitutes correct routing and justify that definition. The evaluation must report against at least one baseline, such as supplying the full tool set to the model unmodified

Deliverables

1. The router and harness, as runnable code, with a README describing how to run everything with a single command.
2. The evaluation, including the query set you constructed and the results.
3. A design note of at most one page covering your approach, your results, and what you would do with more time.

Submission as a private/public Github repo is preferable.

Evaluation criteria

In order of weight:

1. The design and rigor of the evaluation: coverage of failure modes, choice of metrics, the presence of a credible baseline, and a defensible notion of correctness. Your choice of metrics and the queries you include are themselves part of what is assessed.
2. The quality of reasoning in the design note: the trade-offs among accuracy, latency, and cost; the treatment of routing errors, whose consequences are not symmetric; and how the approach behaves when tool descriptions are sparse, inconsistent, or misleading, or when servers are unknown in advance.
3. The working prototype: that it runs, generalizes beyond the three example servers, and produces a measurable improvement over the baseline.

A note on expectations

Tool routing at this general scale is, by and large, an unsolved problem; there is no known approach that handles arbitrary servers cleanly, and we do not expect one here.

We are not evaluating you against a perfect solution. We are looking for sound harness-design judgment and a clear account of the reasoning behind your choices - what you optimized for, what you traded away, where your approach breaks, and what you would try next. A modest router accompanied by sharp reasoning and a thoughtful evaluation is a stronger submission than an elaborate one presented without justification.